جزوه دوره مهندسی داده و مهندسی هوش مصنوعی

Data Engineering & AI Engineering

یادداشت‌های آموزشی، مفاهیم فنی و تمرین‌های دوره



جلسه دهم : رضا امینی
مدرس :مصطفی قدیمی
تاریخ :جمعه ۱۵ آبان ۱۴۰۵

تاریخ به روز رسانی : ۲۰۲۶۱۱۰۹

درباره این جزوه

این جزوه با هدف ثبت، سازمان‌دهی و بازنویسی آموزشی مطالب دوره رایگان «مهندسی داده و مهندسی هوش مصنوعی» تهیه شده است.

محتوای آن بر اساس توضیحات مدرس، مثال‌های مطرح‌شده در کلاس و پرسش‌وپاسخ‌های آموزشی تدوین می‌شود. تلاش شده است مطالب گفتاری کلاس به متنی منظم، قابل فهم و مناسب مطالعه مستقل تبدیل شوند، بدون اینکه مفهوم اصلی صحبت‌های مدرس تغییر کند.

هدف این جزوه صرفاً خلاصه‌کردن جلسات نیست؛ بلکه ایجاد یک منبع آموزشی است که دانشجو بتواند در طول دوره و حتی چند ماه پس از پایان آن، برای یادگیری دوباره مفاهیم، مرور نکات فنی و آماده‌شدن برای فعالیت‌های عملی به آن مراجعه کند.

رویکرد آموزشی و شیوه تدوین

در تدوین مطالب، ابتدا مفاهیم اصلی هر جلسه با زبانی ساده و دقیق توضیح داده می‌شوند. سپس مثال‌های مدرس، کاربردهای عملی و ارتباط موضوعات با مسائل واقعی مهندسی داده و هوش مصنوعی بررسی می‌شوند.

برای حفظ دقت علمی و تمایز میان محتوای اصلی کلاس و توضیحات تکمیلی، از ساختار زیر استفاده شده است:

- **مطالب اصلی جلسه :**بازنویسی منظم توضیحات مدرس با حفظ مفهوم و مثال‌های اصلی.
- **تکمیل فنی :**توضیحات دقیق‌تر، اصلاح برداشت‌های احتمالی و نکات تکمیلی بر اساس منابع فنی معتبر.
- **مثال اضافی :**مثال‌ها و سناریوهایی که برای درک بهتر مفاهیم به جزوه افزوده شده‌اند.
- **نکات کاربردی :**ملاحظات اجرایی، محدودیت‌ها و تجربه‌های مرتبط با استفاده عملی از مفاهیم.
- **جمع‌بندی و سوال‌های مرور :**مرور مهم‌ترین مطالب و ارزیابی میزان یادگیری در پایان هر جلسه.

اصطلاحات تخصصی مانندData Pipeline ، Data Warehouse، ETL، Orchestrationو سایر مفاهیم فنی با نام اصلی انگلیسی حفظ شده‌اند تا مطالعه مستندات رسمی و منابع تخصصی برای دانشجویان آسان‌تر باشد.

محدوده و توسعه جزوه

این مجموعه به‌صورت تدریجی و هم‌زمان با انتشار جلسات جدید تکمیل می‌شود و دو مسیر اصلی دوره، یعنی Data Engineering وAI Engineering ، را پوشش می‌دهد.

نسخه‌های به‌روزشده جزوه‌ها، منابع تکمیلی و یادداشت‌های هر جلسه از طریق مخزن عمومی GitHub در دسترس خواهند بود.

مخزن جزوه‌ها:

https://github.com/LaneZero/data-ai-engineering-course

وب‌سایت اصلی دوره:

https://mostafaghadimi.github.io/data-course/

یادداشت درباره اعتبار محتوا

این جزوه یک مجموعه آموزشی مستقل و غیررسمی است و وابستگی رسمی به مدرس یا برگزارکنندگان دوره ندارد.

حقوق محتوای اصلی دوره متعلق به صاحبان آن است. توضیحات تکمیلی و مثال‌های افزوده‌شده نیز لزوماً بیانگر دیدگاه یا تأیید مدرس نیستند.

با وجود تلاش برای حفظ دقت فنی، توصیه می‌شود هنگام اجرای پروژه‌های واقعی، اطلاعات حساس یا تصمیم‌گیری‌های فنی مهم، مستندات رسمی ابزارها و منابع اصلی نیز بررسی شوند.

هدف از انتشار این مجموعه، **کمک به یادگیری عمیق‌تر، مرور منظم مفاهیم و اشتراک دانش با علاقه‌مندان حوزه داده و هوش مصنوعی** است.

فهرست مطالب

۱. هدف و محدوده جلسه ۳

۲. چرا Airflow و نه فقط Cron یا اسکریپت؟ ۳

۳. مفهوم DAG و اجزای اصلی ٤

۴. معماری Airflow و تفاوت نسخه‌ها ٤

۵. اولین DAG: یک گردش‌کار با چهار Task ٥

۶. XCom و TaskFlow API ٦

۷. زمان‌بندی، Data Interval و Catchup ۷

۸. Params و الگوی اجرای دستی ۸

۹. Dynamic Task Mapping, Branching و TaskGroup ۹

۱۰. Connection, Variable, Hook و Operator ۱۰

۱۱. Sensors: انتظار برای رخداد ۱۱

۱۲. Retry, Callback, Timeout و پایدارسازی ۱۱

۱۳. Setup و Teardown: پاک‌سازی حتی پس از شکست ۱۳

۱۴. خطاهای رایج و شیوه عیب‌یابی ۱۳

۱۵. تمرین عملی مرحله‌به‌مرحله ۱٤

۱۶. چک‌لیست پیش از انتشار DAG ۱٤

۱۷. واژه‌نامه ۱٥

۱۸. سوال‌های مرور و پاسخ کوتاه ۱٦

۱۹. منابع ۱۷

۱. هدف و محدوده جلسه

این جلسه یک کارگاه عملی درباره Apache Airflow است؛ ابزاری برای تعریف، زمان‌بندی، هماهنگ‌کردن و پایش وظایف یک پایپ‌لاین. مدرس در محیط **Airflow 2.9** کار کرد و از مثال‌های ساده به سراغ XCom، زمان‌بندی، Dynamic Task Mapping, Branching, Connection, Sensor و سرانجام Retry و Cleanup رفت. این جزوه سخنان مدرس را با نظم آموزشی بازسازی می‌کند.

پس از این جلسه باید بتوانید تفاوت زمان‌بندی صرف و ارکستریشن را توضیح دهید، یک DAG بسازید، وابستگی‌ها را تعیین کنید، وضعیت Task Instance را بخوانید، بازاجرای تاریخی را مدیریت کنید و یک مسیر آزمایشی را برای شکست و بازیابی طراحی کنید.

۲. چرا Airflow و نه فقط Cron یا اسکریپت؟

مدرس بحث را با پرسش رایج «چرا به‌جای Airflow چند اسکریپت و Cron ننویسیم؟» آغاز کرد. وقتی اجرای یک کار به کارهای قبلی وابسته باشد، شکست آن نیاز به هشدار داشته باشد، اجرای تاریخی لازم شود یا مسئول پایپ‌لاین بخواهد به‌سادگی Logها و وضعیت هر مرحله را ببیند، مدیریت با مجموعه‌ای از Cron Jobهای مستقل پیچیده می‌شود. Airflow امکانات مشخصی برای این نیازها دارد.

نیاز	اسکریپت و Cron ساده	Airflow
زمان‌بندی	امکان‌پذیر	زمان‌بندی در سطح DAG
وابستگی بین مراحل	نیازمند پیاده‌سازی اضافه	Dependency و Trigger Rule
مشاهده وضعیت	معمولاً بررسی Log جداگانه	UI, Task Instance و Log
خطا و تلاش مجدد	نیازمند منطق اختصاصی	Retry و Callback
اجرای تاریخ‌های عقب‌افتاده	نیازمند طراحی مستقل	Catchup و Backfill
نگهداری اتصال‌ها	راهکار خارجی	Connection و Secret Backend

- **تکمیل فنی:** Airflow جایگزین ابزارهای پردازش داده مثل Spark یا موتور دیتابیس نیست؛ وظیفه اصلی آن **Orchestration** است، نه پردازش سنگین داخل Scheduler یا Worker. Cron هم می‌تواند با اسکریپت و ابزارهای جانبی بخشی از نیازهای جدول را پوشش دهد. انتخاب ابزار به پیچیدگی واقعی پروژه وابسته است.

۳. مفهوم DAG و اجزای اصلی

DAG مخفف Directed Acyclic Graph یعنی «گراف جهت‌دار بدون دور» است. هر گره نماینده یک Task است و یال‌ها مشخص می‌کنند کدام کار باید پیش از کار دیگر انجام شود. «بدون دور» بودن مهم است: A نمی‌تواند به B وابسته باشد و B نیز در همان DAG به A وابسته بماند.

```
extract → validate → transform → load
```

اصطلاح	معنی کاربردی
DAG	تعریف کلی گردش‌کار و وابستگی‌ها
Task	یک واحد کار در DAG
Operator	قالب قابل استفاده مجدد برای ساخت Task
DAG Run	یک نوبت اجرای یک DAG با شناسه و بازه مشخص
Task Instance	اجرای یک Task در یک DAG Run مشخص
XCom	انتقال مقدارهای کوچک بین Taskها
Executor / Worker	اجزای تحویل و اجرای کارها

مدرس وضعیت‌هایی مانند `queued`، `running`، `success`، `failed` و `skipped` را در UI نشان داد. تفاوت تعریف DAG با **یک نوبت اجرای آن** مهم است؛ یک DAG ثابت می‌تواند ده‌ها DAG Run داشته باشد و هر Run نیز Task Instanceهای مستقل دارد.

- **تکمیل فنی:** Operator خودِ «Task Instance» نیست؛ Operator یا TaskFlow Function هنگام تعریف DAG, Task می‌سازد و در هر اجرا یک Instance با State جداگانه ایجاد می‌شود. DAG همچنین کد Python است، ولی هر فایل Python الزاماً یک DAG قابل شناسایی تولید نمی‌کند.

۴. معماری Airflow و تفاوت نسخه‌ها

در توضیح معماری نسخه ۲.۹، مدرس به پوشه `dags/`، پردازش و Parse فایل‌ها، Metadata Database, Scheduler, Executor/Worker و Webserver اشاره کرد. Parser فایل‌های DAG را می‌خواند؛ Scheduler تشخیص می‌دهد کدام Task آماده است؛ Executor و Worker کارها را اجرا می‌کنند؛ Metadata Database وضعیت اجراها، تنظیمات و اطلاعات عملیاتی را نگه می‌دارد؛ UI مشاهده و مدیریت را آسان می‌کند.

```
Python DAG files → DAG Processor → Metadata DB
                                      ↕
                              Scheduler → Executor → Worker
                                      ↕
                                 Webserver / UI
```

- **تکمیل فنی (نسخه ۳):** معماری Airflow 3 نسبت به ۲.۹ تغییر مهم‌تری از «فقط ظاهر UI» دارد. Task و Worker دیگر مستقیماً مثل الگوی قبلی به Metadata DB دسترسی ندارند و از مسیر Task Execution API / API Server استفاده می‌کنند. در نسخه ۳ همچنین Task SDK و مدیریت نسخه‌های DAG مطرح است. **نمونه‌های این جزوه با APIهای ۲.۹ نوشته شده‌اند**؛ برای مهاجرت باید Imports و سازگاری Providerها بازبینی شوند.

۵. اولین DAG: یک گردش‌کار با چهار Task

مدرس یک DAG بدون Schedule نشان داد که با Start آغاز می‌شد، دو مسیر Bash و Python را طی می‌کرد و سپس به End می‌رسید. این نمونه برای فهم رسم گراف و بررسی Logهاست.

مثال اضافی | سازگار با Airflow 2.9: فایل زیر را مثلاً با نام `hello_workshop.py` در پوشه `dags` بگذارید. این کد نمونه آموزشی است؛ روی محیط Airflow نصب‌شده اجرا می‌شود و نیازی به دیتابیس عملیاتی ندارد.

```python
from datetime import datetime
from airflow.decorators import dag, task
from airflow.operators.bash import BashOperator
from airflow.operators.empty import EmptyOperator

@dag(
    dag_id="workshop_hello",
    start_date=datetime(2026, 11, 6),
    schedule=None,  # Run only when manually triggered.
    catchup=False,
    tags=["databurst", "training"],
)
def workshop_hello():
    start = EmptyOperator(task_id="start")
    bash = BashOperator(task_id="hello_bash", bash_command="echo
 Hello")

    @task
    def hello_python():
        # Small function executes as a TaskFlow task.
```

```
        print("Hello from Python")

    end = EmptyOperator(task_id="end")
    start >> [bash, hello_python()] >> end

workshop_hello()
```

پس از Trigger کردن DAG در UI می‌توان Grid/Graph و Log هر Task را دید. دو شاخه میانی می‌توانند مستقل از یکدیگر اجرا شوند و End به نتیجه هر دو وابسته است.

- **تکمیل فنی:** تنظیم `schedule=None` به معنی اجرا نشدن خودکار بر اساس زمان است، نه ممنوع‌بودن اجرای دستی. همچنین اجرای هم‌زمان واقعی تابع Executor، ظرفیت Worker, Pool و تنظیمات هم‌زمانی است.

۶. XCom و TaskFlow API

مدرس یک گردش‌کار `extract → transform → load → notify` نشان داد. در TaskFlow API بازگرداندن مقدار از یک تابع `@task` می‌تواند آن مقدار را در XCom ثبت کند و دادن خروجی همان تابع به Task بعدی، هم انتقال مقدار و هم وابستگی را تعریف کند.

مثال اضافی: داده صرفاً ساختگی و کوچک است و نمایش یک Pipeline واقعی را شبیه‌سازی می‌کند.

```
from datetime import datetime
from airflow.decorators import dag, task

@dag(dag_id="workshop_xcom", start_date=datetime(2026, 11, 6),
     schedule=None, catchup=False)
def xcom_demo():
    @task
    def extract():
        # Compact values can travel through XCom.
        return {"EUR": 19, "USD": 80}

    @task
    def transform(amounts: dict):
        return {code: value * 2 for code, value in amounts.items
()}

    @task
    def load(amounts: dict):
        # In a real pipeline use an idempotent database write.
        print(amounts)
```

```
        return len(amounts)

    @task
    def notify(count: int):
        print(f"Processed {count} currencies")

    notify(load(transform(extract())))

xcom_demo()
```

- **تکمیل فنی:** XCom برای جابه‌جایی DataFrameهای بزرگ یا فایل‌های حجیم مناسب نیست. بهتر است داده در Database یا Object Storage باقی بماند و فقط آدرس، Object Key یا Metadata کوچک منتقل شود. نوع داده باید با XCom Backend و Serializer سازگار باشد. همچنین خروجی Log ممکن است حاوی اطلاعات حساس باشد؛ Token و رمز را در خروجی Task چاپ نکنید.

۷. زمان‌بندی، Data Interval و Catchup

برای زمان‌بندی می‌توان از `@daily` یا عبارت Cron مانند `0 0 * * *` استفاده کرد. مدرس تفاوت «زمان واقعی اجرا» و «بازه داده‌ای که Run نمایندگی می‌کند» را با مثال اجرای روزانه توضیح داد. یک اجرای برنامه‌ریزی‌شده برای داده‌های روز قبل ممکن است بعد از پایان بازه آن روز آغاز شود.

پارامتر	مفهوم
`schedule`	الگوی ساخت DAG Run
`start_date`	نقطه شروع منطقی برنامه‌ریزی
`catchup=True`	ایجاد Run برای بازه‌های برنامه‌ریزی‌شده گذشته که هنوز اجرا نشده‌اند
`data_interval_start`	ابتدای بازه داده Run
`data_interval_end`	انتهای بازه داده Run
`logical_date`	شناسه زمانی منطقی Run، نه الزاماً زمان شروع واقعی

مثال اضافی: برای اجرای روزانه، یک Task می‌تواند با `data_interval_start` و `data_interval_end` داده همان بازه را پردازش کند. این کار برای اجرای مجدد تاریخی از استفاده از `datetime.now()` قابل اعتمادتر است.

```python
from airflow.decorators import task

@task
# Airflow injects runtime context into a TaskFlow function.
def print_interval(**context):
    start = context["data_interval_start"]
    end = context["data_interval_end"]
    print(f"Window: [{start.isoformat()}, {end.isoformat()})")
```

- **تکمیل فنی:** `catchup=True` می‌تواند تعداد زیادی DAG Run بسازد؛ قبل از فعال‌کردن آن، هزینه محاسبات و **Idempotency** مقصد را بررسی کنید. قالب `{{ ds }}` در Airflow معمولاً نمایش تاریخ **منطقی Run** است، نه لزوماً «امروزِ سیستم»؛ جایگزینی Jinja عموماً در مرحله Render فیلد Template انجام می‌شود، نه لزوماً هنگام Parse فایل DAG.

۸. Params و الگوی اجرای دستی

مدرس نشان داد که می‌توان پارامترهای پیش‌فرض برای DAG تعریف کرد و هنگام Manual Trigger مقدار متفاوتی وارد کرد؛ مثلاً به‌جای پردازش یک روز، ده روز داده بررسی شود. این روش کمک می‌کند منطق اصلی Pipeline ثابت بماند و فقط دامنه اجرا تغییر کند.

مثال اضافی: نمونه‌ای از ثبت تنظیمات قابل تغییر؛ برای کار حساس، `params` را اعتبارسنجی کنید و از آن برای ساخت مستقیم SQL استفاده نکنید.

```python
from airflow.models.param import Param
from airflow.decorators import dag, task
from datetime import datetime

@dag(
    dag_id="workshop_params",
    start_date=datetime(2026, 11, 6),
    schedule=None,
    params={"days_back": Param(1, type="integer", minimum=1, maximum=3
0)},
)
def params_demo():
    @task
    def show_days(**context):
        days = context["params"]["days_back"]
        print(f"Process the previous {days} day(s)")
    show_days()

params_demo()
```

۹. Dynamic Task Mapping, Branching و TaskGroup

مدرس سناریویی مطرح کرد که تعداد فایل‌های ورودی هر روز متفاوت است. به‌جای ساختن دستی چهار یا ده Task، **Dynamic Task Mapping** با `expand` برای هر عضو لیست، یک Task Instance جدا ایجاد می‌کند. `partial` برای پارامتر ثابت به کار می‌رود.

> **مثال اضافی:** این مثال فقط منطق تقسیم کار را نشان می‌دهد و فایل واقعی را تبدیل نمی‌کند.

```
from datetime import datetime
from airflow.decorators import dag, task

@dag(dag_id="workshop_mapping", start_date=datetime(2026, 11, 6)
,
     schedule=None)
def mapping_demo():
    @task
    def list_files():
        return ["orders.csv", "customers.csv", "payments.csv"]

    @task
    def process_file(filename: str, target_format: str):
        # Example only: no physical file is read or written.
        print(f"Convert {filename} to {target_format}")
        return len(filename)

    @task
    def summarize(lengths: list[int]):
        print(f"Total filename length: {sum(lengths)}")

    results = process_file.partial(target_format="parquet").expa
nd(
        filename=list_files()
    )
    summarize(results)

mapping_demo()
```

Branching برای انتخاب یک مسیر از چند مسیر است: اگر حجم داده کم باشد مسیر سبک و اگر زیاد باشد مسیر پردازش بزرگ انتخاب شود. در Demo انتخاب شاخه با بازگرداندن `task_id` انجام شد؛ مسیر دیگر به حالت `skipped` رفت. برای اتصال دوباره شاخه‌ها، یک Trigger Rule مناسب لازم است؛ وگرنه `all_success` ممکن است Task انتهایی را هم Skip کند.

قابلیت	کاربرد
`expand`	ساخت Task Instanceهای متناسب با ورودی
`partial`	دادن پارامتر ثابت به Task Mapشده
`@task.branch`	انتخاب مسیر و Skip کردن مسیر دیگر
`TaskGroup`	گروه‌بندی Taskها برای خوانایی DAG و UI
`TriggerRule.NONE_FAILED_MIN_ONE_SUCCESS`	ادامه بعد از شاخه‌ای که یک مسیر موفق و مسیر دیگر Skip شده است

- **تکمیل فنی:** `TaskGroup` به‌تنهایی یک DAG مستقل یا Worker جدا نیست؛ فقط گروه‌بندی Taskها در همان DAG است. نتیجه Mapping می‌تواند تعداد زیادی Instance تولید کند، پس محدودیت‌های هم‌زمانی و هزینه را در نظر بگیرید.

۱۰. Connection, Variable, Hook و Operator

در بخش بعد، مدرس نشان داد مشخصات اتصال به PostgreSQL یا سامانه‌های دیگر را یک‌بار به‌صورت Connection تعریف کنیم و در DAGها فقط `conn_id` بدهیم. Variable برای مقادیر پیکربندی و Hook برای کار برنامه‌نویسی با سرویس خارجی معرفی شد. Operator اجرای یک عمل مشخص را در قالب Task آماده می‌کند.

مفهوم	مثال استفاده	احتیاط
Connection	مشخصات Host, Port, User و احراز هویت	استفاده از Secret Backend و دسترسی محدود
Variable	نام مقصد یا تنظیم عمومی Pipeline	برای اطلاعات حساس ترجیحاً Secret Backend
Hook	دریافت Client برای PostgreSQL	مدیریت تراکنش و بستن Connection
SQL Operator	اجرای Query با Connection ID	Query پارامتری و کنترل اثر تغییرات

- **تکمیل فنی:** قرارگرفتن Connection در Metadata Database لزوماً به معنی حفاظت کامل Secretها نیست. دسترسی UI، رمزنگاری، کلیدها، Secret Backend و Permissionها باید طبق سیاست امنیتی تنظیم شوند. همچنین نام Import و کلاس SQL Operatorها میان نسخه Providerها تغییر می‌کند؛ کد روی اسلاید بدون دانستن نسخه Provider نباید عیناً کپی شود.

۱۱. Sensors: انتظار برای رخداد

Sensor منتظر برقرارشدن شرطی می‌ماند؛ مثلاً رسیدن یک فایل، آماده‌شدن داده یک جدول یا پایان عملیات سرویس دیگر. مدرس سه رویکرد انتظار را توضیح داد:

شیوه	رفتار	اثر بر منابع
`poke`	Task در حالت انتظار می‌ماند و دوره‌ای شرط را بررسی می‌کند	نگهداری Slot در طول انتظار
`reschedule`	میان بررسی‌ها Slot Worker را آزاد می‌کند	مناسب انتظار طولانی‌تر در شرایط متعارف
Deferrable	Task با Triggerer منتظر رخداد می‌ماند	مناسب بسیاری از انتظارهای طولانی؛ وابسته به Operator/Provider

در Demo یک مرحله ۳۰ ثانیه صبر می‌کرد و مرحله‌ای دیگر تا ایجاد فایل مشخص منتظر می‌ماند. فرق Sensor با Schedule این است که Schedule **زمان آغاز بررسی/اجرا** را تعیین می‌کند، اما Sensor **آمادگی وابستگی خارجی** را ارزیابی می‌کند.

> **نیازمند بررسی:** مسیر واقعی فایلِ Sensor، پیکربندی Worker و نسخه Provider در Transcript مشخص نیست؛ اجرای نمونه‌ای که منتظر فایل است، بدون هماهنگی مسیر و دسترسی فایل بین Workerها لزوماً موفق نمی‌شود.

۱۲. Retry, Callback, Timeout و پایدارسازی

مدرس در جمع‌بندی به نکات تولیدی پرداخت: Retry برای خطاهای گذرا، افزایش فاصله تلاش‌ها، Callback هنگام شکست، محدودکردن اجرای هم‌زمان یک DAG, Timeout و قابلیت اجرای دوباره بدون خراب‌کردن داده‌ها.

گزینه	هدف
`retries`	تعداد تلاش‌های مجدد پس از شکست
`retry_delay`	فاصله بین Retryها
`retry_exponential_backoff`	افزایش تدریجی فاصله تلاش‌ها

گزینه	هدف
`on_failure_callback`	اطلاع‌رسانی یا ثبت رخداد شکست
`max_active_runs=1`	جلوگیری از هم‌پوشانی بیش از یک DAG Run فعال
`execution_timeout`	محدودکردن زمان Task

مثال اضافی | الگوی قابل بازاجرا: در کد فرضی زیر ابتدا داده روز مشخص جایگزین می‌شود. در سامانه واقعی باید حذف و درج در یک تراکنش مناسب یا با الگوی Upsert/Merge ایمن انجام شوند تا شکست بین دو مرحله موجب ازدست‌رفتن داده نشود.

```
-- Example only: execute in a transaction supported by your database.
BEGIN;
DELETE FROM daily_sales WHERE business_date = :business_date;
INSERT INTO daily_sales (business_date, total_amount)
SELECT business_date, SUM(amount)
FROM staging_sales
WHERE business_date = :business_date
GROUP BY business_date;
COMMIT;
```

- **تکمیل فنی:** `ROLLBACK` هنگام خطا، کنترل قفل و Isolation Level و حفظ یکتایی باید در پیاده‌سازی واقعی مشخص باشند؛ صرف «اول پاک‌کردن بعد درج» بدون Transaction به‌معنای Idempotency ایمن نیست. `max_active_runs=1` نیز به‌تنهایی تضمین نمی‌کند Pipelineهای دیگر به همان جدول دست نزنند.

۱۳. Setup و Teardown: پاک‌سازی حتی پس از شکست

در مثال آخر، مدرس یک Task برای ساخت منبع موقت، Taskهای اصلی برای استخراج و بارگذاری و Task پاک‌سازی معرفی کرد. هدف این بود که حتی اگر مرحله اصلی شکست بخورد، منبع موقت باقی نماند و اجرای بعدی دچار تداخل نشود.

```
setup_resource → extract → load → check
       └──────────────────────→ teardown_resource
```

- **تکمیل فنی:** در Airflow 2.9 قابلیت رسمی Setup/Teardown با APIهای `as_setup()` و `as_teardown()` وجود دارد. Teardown در صورت موفقیت Setup و شکست کارهای اصلی نیز می‌تواند اجرا شود؛ اما نباید فرض کرد در هر نوع خرابی زیرساخت یا قطعی کامل سیستم، Cleanup حتماً انجام شده است. بهتر است عملیات Cleanup تکرارپذیر باشد و منابع رهاشده پایش شوند.

۱۴. خطاهای رایج و شیوه عیب‌یابی

نشانه	علت احتمالی	بررسی پیشنهادی
DAG در UI ظاهر نمی‌شود	خطای Import/Parse یا مشکل مسیر DAG	Import Error و Log پردازنده DAG
Task روی `queued` می‌ماند	ظرفیت Worker, Pool یا Executor	صف و تنظیمات هم‌زمانی
اجرای تاریخی داده امروز را می‌خواند	استفاده از ساعت سیستم	`data_interval_start/end` و Logical Date
Task بعد از Branch اجرا نمی‌شود	Skip شدن شاخه و Trigger Rule نامناسب	State بالادستی و Trigger Rule
خطای Serialization XCom می‌دهد	داده ناسازگار یا بزرگ	اندازه و نوع Payload
Sensor همیشه منتظر است	فایل یا شرط در محیط Worker دیده نمی‌شود	مسیر، Permission و Connection
بازاجرای Load داده را دوبرابر می‌کند	نبود Idempotency	Upsert، کلید یکتا، Transaction و Partition

۱۵. تمرین عملی مرحله‌به‌مرحله

۱. یک DAG بدون Schedule با Taskهای Start و End و دو Task موازی بسازید. Graph View را مشاهده کنید و ترتیب اجرا را توضیح دهید.

۲. یک TaskFlow DAG با سه Task بسازید: `extract` یک دیکشنری کوچک بازگرداند، `transform` آن را تبدیل کند و `load` نتیجه را صرفاً چاپ کند. XCom و Log هر Task را بررسی کنید.

۳. با `schedule='0 6 * * *'` یک DAG روزانه تعریف کنید؛ توضیح دهید هر Run مربوط به کدام Data Interval است. در محیط آزمایشی رفتار `catchup` را مقایسه کنید.

۴. فهرست سه نام فایل ساختگی را با Dynamic Mapping بین سه Instance پردازش کنید و نتیجه‌ها را جمع بزنید.

۵. یک Branch با دو مسیر بسازید. مسیر انتخاب‌نشده را در UI پیدا کنید و Trigger Rule تسک اتصال را تنظیم کنید.

۶. یک Task عمداً خطای موقت ایجاد کند، `retries=2` بگیرد و نتیجه Retryها را در Log ببینید. سپس طراحی امن برای پاک‌سازی منبع موقت را توضیح دهید.

- **تکمیل فنی:** تمرین‌ها را ابتدا در محیط محلی یا آزمایشی انجام دهید. هرگز Credential واقعی را در Git، متن DAG, Log یا XCom ذخیره نکنید و برای تمرین به پایگاه داده تولیدی دسترسی نوشتن ندهید.

۱۶. چک‌لیست پیش از انتشار DAG

- DAG بدون خطای Parse بارگذاری می‌شود و Schedule و Timezone روشن‌اند.
- هر Task یک مسئولیت مشخص دارد و ورودی/خروجی آن معلوم است.
- Data Interval برای اجرای عقب‌افتاده درست استفاده می‌شود.
- Retry و Timeout و Alert برای کار حساس تنظیم شده‌اند.
- Load در برابر اجراهای تکراری و هم‌زمان مقاوم است.
- داده حجیم از XCom عبور نمی‌کند و Secret چاپ نمی‌شود.
- شکست و موفقیت هر مسیر Branch و Cleanup آزمایش شده‌اند.
- حجم Taskهای Mapشده و ظرفیت Worker محدود شده‌اند.

۱۷. واژه‌نامه

اصطلاح	شرح کوتاه
Orchestration	هماهنگ‌سازی اجرای فعالیت‌های وابسته
DAG	گراف جهت‌دار بدون دور
Task	واحد اجرایی در گردش‌کار
Operator	قالب تعریف Task
DAG Run	یک اجرای خاص DAG
Task Instance	اجرای مشخص یک Task
Scheduler	زمان‌بند و تصمیم‌گیر اجرای Task
Executor	واسط تحویل کار برای اجرا
Worker	اجراکننده Task
Metadata DB	پایگاه اطلاعات وضعیت Airflow
XCom	انتقال مقدار کوچک میان Taskها
TaskFlow API	تعریف Task با Decoratorهای Python
Data Interval	بازه داده وابسته به Run
Logical Date	تاریخ منطقی اجرای برنامه‌ریزی‌شده
Catchup	تولید اجرای بازه‌های عقب‌افتاده
Backfill	بازاجرای تاریخ‌های گذشته
Cron	قالب بیان زمان‌بندی
Dynamic Task Mapping	ساخت Instance بر اساس ورودی
Branching	انتخاب یک یا چند مسیر اجرایی
TaskGroup	گروه‌بندی Taskها
Trigger Rule	شرط فعال‌شدن Task بر اساس بالادستی‌ها
Connection	مشخصات اتصال به سرویس
Hook	Client یا رابط Python اتصال به سرویس
Sensor	انتظار برای تحقق شرط
Deferrable	انتظار بدون اشغال دائمی Slot Worker
Retry	تلاش مجدد پس از شکست
Callback	تابع واکنش به رویداد Task

Idempotency	حفظ نتیجه درست در اجرای تکراری
Setup/Teardown	ساخت و جمع‌آوری منبع موقت

۱۸. سوال‌های مرور و پاسخ کوتاه

- **چرا Cron به‌تنهایی برای یک ETL چندمرحله‌ای کافی نیست؟** زیرا مدیریت وابستگی، وضعیت، Retry و اجرای تاریخی نیاز به منطق افزوده دارد.
- **DAG چرا بدون دور است؟** برای جلوگیری از وابستگی چرخه‌ای و تعریف ترتیب قابل اجرا.
- **فرق DAG و DAG Run چیست؟** اولی تعریف گردش‌کار است و دومی یک اجرای مشخص از آن.
- **فرق Task و Task Instance چیست؟** Task تعریف کار است؛ Instance اجرای آن در یک Run.
- **XCom برای چه چیزی مناسب‌تر است؟** شناسه، مسیر فایل، آمار یا مقدار کوچک؛ نه DataFrame حجیم.
- **آیا `schedule=None` اجرای دستی را می‌بندد؟** خیر؛ زمان‌بندی خودکار را غیرفعال می‌کند.
- **چرا `datetime.now()` برای Backfill مسئله‌ساز است؟** چون تاریخ اجرای واقعی را می‌دهد نه بازه داده مورد نظر.
- **Catchup چه خطری دارد؟** ایجاد Runهای زیاد و مصرف منابع یا ثبت تکراری داده.
- **`expand` چه می‌کند؟** بر اساس فهرست ورودی Instanceهای متعدد می‌سازد.
- **Branch انتخاب‌نشده چه State می‌گیرد؟** معمولاً `skipped`.
- **چرا Task پس از Branch ممکن است Skip شود؟** چون Trigger Rule پیش‌فرض به موفقیت همه بالادستی‌ها وابسته است.
- **تفاوت Connection و Variable چیست؟** اولی تنظیمات اتصال و دومی پارامتر عمومی ذخیره می‌کند.
- **برای انتظار طولانی کدام Sensor مناسب‌تر است؟** با توجه به Provider، حالت Deferrable یا Reschedule معمولاً کم‌هزینه‌تر از Poke دائمی است.
- **چرا Retry باید با Idempotency همراه باشد؟** چون تکرار Load نباید داده را دوباره اضافه یا خراب کند.
- **Setup/Teardown چه هدفی دارد؟** مدیریت چرخه زندگی منابع موقت و تلاش برای پاک‌سازی پس از اجرای کارها.

۱۹. منابع

نمونه‌های کد با برچسب «مثال اضافی» ساخته شده‌اند. توضیحات نسخه‌ای و نکات رفتاری از مستندات رسمی زیر بررسی شده‌اند:

- Apache Airflow 2.9.3, DAGs: https://airflow.apache.org/docs/apache-airflow/2.9.3/core-concepts/dags.html
- Apache Airflow 2.9.3, Tasks: https://airflow.apache.org/docs/apache-airflow/2.9.3/core-concepts/tasks.html
- Apache Airflow 2.9.3, Setup and Teardown: https://airflow.apache.org/docs/apache-airflow/2.9.3/howto/setup-and-teardown.html
- Apache Airflow 3, Upgrading to Airflow 3: https://airflow.apache.org/docs/apache-airflow/stable/installation/upgrading_to_airflow3.html